

Overspeed Vehicle Detection System

An AI-based traffic monitoring system that detects vehicles from surveillance videos, tracks them, estimates their speed, and identifies overspeeding vehicles using deep learning and computer vision.

The project also includes a web-based interface that allows users to upload traffic videos and initiate automated vehicle detection and speed analysis.

Project Overview

The Overspeed Vehicle Detection System is designed to automate vehicle monitoring from traffic surveillance footage.

The system:

- Detects vehicles in traffic videos
- Tracks detected vehicles using unique IDs
- Estimates vehicle speed from frame-based movement
- Identifies vehicles exceeding the configured speed threshold
- Generates overspeed alerts
- Provides a web interface for video upload and processing
- Produces processed detection results

Objectives

- Automate vehicle detection from traffic surveillance videos
- Monitor vehicle movement across video frames
- Estimate the speed of detected vehicles

- Identify overspeeding vehicles
- Provide a simple web interface for video processing
- Demonstrate the integration of AI/computer vision with web technologies

Tech Stack

AI / Computer Vision

- Python
- YOLOv8 Nano
- OpenCV
- PyTorch
- NumPy

Web Technologies

- HTML
- CSS
- JavaScript
- React.js
- Flask

Development Tools

- VS Code
- Git / GitHub

System Workflow

Traffic Video



Video Upload



Frame-by-Frame Processing



Vehicle Detection using YOLOv8



Vehicle Tracking



Speed Estimation



Speed Threshold Comparison



Overspeed Detection



Detection Results / Output

Model

YOLOv8 Nano

The system uses the YOLOv8 Nano model for vehicle detection.

YOLOv8n is a lightweight object detection model suitable for applications where efficient inference is important.

The model is used to identify vehicles within individual video frames.

Detection Classes

The system focuses on common road vehicles including:

- Car
- Motorcycle
- Bus
- Truck

Vehicle Detection

Each processed frame is passed through the YOLOv8 model.

For every detected vehicle, the system obtains:

- Bounding box coordinates
- Object class
- Confidence score
- Vehicle ID

Bounding boxes are drawn around detected vehicles in the processed output.

Vehicle Tracking

A simple center-point based tracking approach is used to associate detected vehicles across frames.

The center of a bounding box is calculated as:

$$cx = (x1 + x2) / 2$$

$$cy = (y1 + y2) / 2$$

A basic ID assignment mechanism is then used to associate detections with vehicle IDs.

This provides a lightweight tracking approach suitable for the prototype.

Speed Estimation

Vehicle speed is estimated based on the movement of a vehicle between video frames.

The basic relationship is:

Speed = Distance / Time

The prototype uses pixel displacement and a scaling factor to estimate speed:

$\text{speed} = (\text{distance_pixels} / \text{time_taken}) \times 0.1$

The estimated speed is then compared with the configured speed threshold.

If the estimated speed exceeds the threshold, the vehicle is identified as speeding.

Overspeed Detection

When a vehicle exceeds the configured speed limit, an overspeed alert is generated.

Example:

Vehicle ID: 0 | Speed: 22.06

OVERSPEEDING → ID: 0, Speed: 22.06

This allows potentially unsafe vehicles to be identified automatically from the processed video.

Web Interface

A web-based interface was added to the vehicle detection system to make the application easier to use.

The interface provides:

- Traffic video selection
- Video upload
- Detection initiation
- Processing status
- Integration with the vehicle detection backend

Application Flow

User



Select Traffic Video



Upload & Detect



Backend Video Processing



Vehicle Detection & Speed Analysis



Overspeed Detection



Results

Features

- AI-based vehicle detection
 - YOLOv8 object detection
 - Vehicle ID assignment
 - Speed estimation
 - Overspeed detection
 - Bounding boxes around detected vehicles
 - Overspeed alerts
 - Traffic video upload
 - Web-based user interface
 - Processed video output
-

Project Structure

vehicle_violation_project/

|

└─ app.py

└─ vehicle.py

└─ requirements.txt

|

└─ templates/

| └─ ...

|

└─ uploads/

```
|
|
├── outputs/
|
└── .gitignore
```

File Description

File / Folder	Description
app.py	Flask application and web interface/backend logic
vehicle.py	Vehicle detection, tracking and speed estimation logic
requirements.txt	Python dependencies
templates/	Web interface templates
uploads/	Temporary uploaded videos
outputs/	Generated processing results

Installation and Setup

1. Clone the Repository

```
git clone <repository-url>
```

```
cd vehicle_violation_project
```

2. Create a Virtual Environment

```
python -m venv .venv
```

Activate the environment on Windows:

```
.venv\Scripts\activate
```

3. Install Dependencies

```
pip install -r requirements.txt
```

4. Download the YOLOv8 Model

Place the YOLOv8 Nano model file in the project directory:

```
yolov8n.pt
```

5. Run the Application

```
python app.py
```

6. Open the Web Interface

Open the following address in your browser:

```
http://127.0.0.1:5000
```

Example Output

The system produces vehicle detection and speed information during processing.

Example:

Vehicle ID: 0 | Speed: 8.15

Vehicle ID: 0 | Speed: 12.93

Vehicle ID: 2 | Speed: 1.85

Vehicle ID: 1 | Speed: 1.98

Vehicle ID: 5 | Speed: 1.72

Vehicle ID: 0 | Speed: 22.06

OVERSPEEDING → ID: 0, Speed: 22.06

The web interface provides feedback while the video is being processed.

Test Cases

Test Case	Input	Expected Result
Valid traffic video	Traffic surveillance video	Video is processed
Multiple vehicles	Video containing multiple vehicles	Vehicles are detected individually
Vehicle movement	Moving vehicle	Vehicle ID is assigned/tracked
Normal speed	Vehicle below threshold	No overspeed alert
Overspeed	Vehicle above threshold	Overspeed alert generated
Video upload	Valid video file	Video accepted for processing
Detection initiation	Uploaded video	Processing starts successfully

Limitations

The current system is a prototype and has several limitations:

- Speed estimation is not calibrated against real-world road measurements.
- The scaling factor is environment-dependent.
- Tracking uses a basic ID-assignment approach.
- Vehicle IDs may not remain consistent in complex traffic scenarios.
- Camera angle and perspective can affect speed estimation.
- No number plate recognition is implemented.
- The system is primarily designed for uploaded video processing rather than production-grade real-time CCTV monitoring.

- Detection accuracy depends on video quality, lighting and camera position.
-

Future Scope

The system can be further improved by implementing:

- Real-world camera calibration for accurate speed estimation
 - Advanced multi-object tracking algorithms
 - Number plate recognition
 - Automatic violation record generation
 - Database integration for storing violations
 - Real-time CCTV stream processing
 - Traffic analytics dashboards
 - Automated email/SMS notifications
 - Cloud deployment
 - Improved handling of night-time and low-quality footage
-

Learning Outcomes

This project provided practical experience in:

- Python programming
- Computer vision
- Object detection using YOLOv8
- Video processing using OpenCV
- Vehicle tracking
- Speed estimation

- Flask-based web application development
 - HTML, CSS and JavaScript
 - React.js frontend development
 - AI/ML application integration
 - Git and GitHub
-

Conclusion

The Overspeed Vehicle Detection System demonstrates the integration of deep learning, computer vision and web technologies for automated traffic monitoring.

The system can detect vehicles from surveillance videos, assign vehicle IDs, estimate their speeds and identify vehicles that exceed the configured speed threshold.

A web-based interface was also integrated to simplify video upload and initiate the detection workflow, providing a practical prototype for AI-assisted traffic monitoring.

References

1. Ultralytics YOLO Documentation
2. OpenCV Documentation
3. Python Documentation
4. PyTorch Documentation
5. Flask Documentation
6. React Documentation
7. NumPy Documentation